

Lexicon

AI-Powered Document Question Answering System

Prompt Engineering, Embeddings, Vector Databases and RAG



Anushay Ayat

SMIT - AI and Data Science

Technical Report, October 2026

Contents

1. Introduction and objectives
2. System architecture
3. Part 1: Prompt engineering
4. Part 2: Embeddings and semantic search
5. Semantic search versus keyword search
6. Implementation details
7. Deployment
8. Testing and limitations
9. Conclusion
10. Appendix: running the project

1. Introduction and objectives

Large language models can write fluent answers, but on their own they know nothing about a person's private documents and they sometimes invent facts. Retrieval-Augmented Generation (RAG) solves both problems: the system first retrieves the passages of the user's documents that are most relevant to a question, and then asks the language model to answer using only those passages.

Lexicon is a complete RAG application built for this assignment. A user uploads PDF, TXT or Markdown files, asks a question in plain language, and receives an answer together with the exact chunks that support it. The project has two goals that mirror the two parts of the assignment:

- **Part 1 (Prompt engineering):** implement zero-shot, few-shot and role-based prompt templates, test them on the same five questions and compare accuracy, clarity and relevance.
- **Part 2 (Embeddings and semantic search):** load at least five documents, split them into chunks, embed them with a sentence-transformer model, store them in ChromaDB, and retrieve the top three most similar chunks with their similarity scores.

The system is written in Python with FastAPI, uses ChromaDB as the vector database, the all-MiniLM-L6-v2 sentence-transformer for embeddings, and Groq-hosted Llama 3.3 70B for text generation. A single-page web interface lets the user upload files, ask questions, switch prompt styles, run the five-question comparison and inspect semantic versus keyword results.

2. System architecture

Every question passes through the same five-stage pipeline. Documents are processed once when they are uploaded (stages 1 to 3); each question then triggers stages 4 and 5.

Stage	What happens	Technology
1. Chunk	Text is cleaned and split into chunks of about 500 characters with 50 characters of overlap, cutting at sentence or word boundaries.	Python (chunking.py)
2. Embed	Each chunk is converted into a 384-dimensional vector that captures its meaning.	all-MiniLM-L6-v2 (ONNX via fastembed)
3. Store	Vectors, chunk text and source metadata are saved in a collection that uses cosine distance.	ChromaDB
4. Retrieve	The question is embedded with the same model and the three nearest chunks are returned with similarity and distance.	ChromaDB query
5. Generate	The chunks are placed in a prompt (zero-shot, few-shot or role-based) and the model writes the answer.	Groq, Llama 3.3 70B

Figure 1. The Lexicon RAG pipeline.

The same embedding model must be used for documents and questions. Vectors from different models live in different spaces and cannot be compared, so the model name is fixed in one configuration file and shared by both paths.

3. Part 1: Prompt engineering

Prompt engineering is the practice of wording the instructions given to a language model so that it produces more accurate, clearer and more useful output. Three techniques were implemented as separate templates in `app/prompts/templates.py`. In every case the retrieved chunks are inserted as the context.

3.1 Zero-shot prompting

The model receives the task and the context but no examples. This is the simplest and cheapest technique and shows the model's default behaviour.

```
Answer the question using the context below.  
  
Context:  
{context}  
  
Question: {question}  
Answer:
```

3.2 Few-shot prompting

Three short worked examples are placed before the real question. They show the expected length and tone, and the third example demonstrates what to say when the documents do not contain the answer, which discourages guessing.

```
Answer the question using only the context. Follow the style of the examples.  
  
Example 1  
Context: Python lists are mutable sequences. Tuples are immutable sequences.  
Question: What is the difference between a list and a tuple?  
Answer: A list can be changed after creation, while a tuple cannot.  
  
Example 2 ... (ChromaDB example)  
Example 3 ... (answer not in the documents)  
  
Now your turn.  
Context: {context}  
Question: {question}  
Answer:
```

3.3 Role-based prompting

A system message assigns the model a professional identity and a short set of rules. The role gives the model a standard to imitate, and the rules make the behaviour explicit and checkable.

```
SYSTEM: You are a senior technical document analyst.
Rules:
1. Use ONLY the provided context. Never invent facts.
2. Start with a one-sentence direct answer, then add short supporting details.
3. Mention the source file names you relied on in brackets.
4. If the context does not contain the answer, say so clearly.

USER: Context: {context}
      Question: {question}
```

3.4 Evaluation method

To keep the comparison fair, all three templates are run on the same five questions and receive exactly the same retrieved chunks, so the only variable is the prompt. The questions cover definitions, process explanations and detail lookups across the five sample documents:

- What is a vector database and why is it used in RAG?
- How are text chunks converted into embeddings?
- What is the difference between a list and a tuple in Python?
- Why is chunk overlap useful?
- What is overfitting in machine learning?

Each answer is scored from 1 to 10 on three criteria by a separate model call that acts as a judge (temperature 0, JSON output). **Accuracy** measures whether the claims are supported by the retrieved context, **clarity** measures how easy the answer is to read and follow, and **relevance** measures whether it addresses the question without drifting. The average of the three is the answer's score. The Compare prompts tab runs the whole experiment in one click and shows every answer next to its scores.

3.5 Results

The experiment was run on the deployed application with the five questions listed above. The answers and the judge scores were both produced by the Groq model `openai/gpt-oss-20b`. Table 1 shows the average of accuracy, clarity and relevance (out of 10) for every answer.

Question	Zero-shot	Few-shot	Role-based
Q1 Vector database and RAG	9.0	9.0	9.3
Q2 Chunks to embeddings	9.0	10.0	9.0
Q3 List vs tuple	10.0	9.3	10.0
Q4 Chunk overlap	10.0	9.7	10.0
Q5 Overfitting	10.0	9.7	10.0
Average (out of 10)	9.60	9.54	9.66

Table 1. Average judge score per question and technique from one run of the Compare prompts tab.

3.6 Analysis: which technique performs best and why

The three techniques are expected to behave differently for reasons that follow from how they steer the model:

- **Zero-shot** relies entirely on the model's default habits. Answers are usually correct when the context is clear, but their length and structure vary from question to question, and the model is more willing to add outside knowledge that is not in the documents.
- **Few-shot** improves consistency because the examples fix the format and length. Its quality depends on the examples chosen, and examples that differ from the real question can bias the answer toward their wording.
- **Role-based** usually gives the best accuracy and clarity in a RAG setting. The explicit rules (use only the context, answer first, cite sources, admit missing information) directly target the main RAG failure modes of hallucination and unverifiable answers, and the source citations let a reader check each claim.

Result. Role-based prompting obtained the highest overall average (9.66), followed by zero-shot (9.60) and few-shot (9.54). Role-based was the best or tied for best on four of the five questions: it led on Q1 (9.3 against 9.0 for both others) and tied zero-shot at 10 on Q3, Q4 and Q5. Few-shot won only Q2 (10 against 9 for the others) and scored lowest on Q3, Q4 and Q5.

How much this proves. The margin is small: 0.12 points separate the best and the worst technique, and every technique scored 9 or higher on every question. With scores this close to the maximum, a single run and an LLM judge, the ranking should be read as a slight advantage for role-based prompting, not as a large or statistically proven difference. That all three scored so high also shows that the retrieval step already supplies strong context, so prompt wording matters mostly at the margins: format, citing sources and refusing to guess.

Why role-based came out ahead. A likely explanation is that its rules (answer only from the context, lead with a direct answer, cite the source files) target exactly the qualities the judge scores: accuracy against the context, clarity and relevance. The few-shot examples fix the style but are generic, so they may have pulled answers toward a short fixed format, although on Q2 that format helped it earn the only 10 in that row. Zero-shot did well because the retrieved context was already strong and the questions were direct.

Limits of the comparison. The judge and the answering model are the same model, which can favour answers written in its own style, and only one run was made. Repeating the run, using a different judge model or adding human review would make the conclusion more reliable. For a real RAG system, role-based prompting is the sensible default because it also adds source citations and an explicit rule for missing information, benefits that a score only partly captures.

4. Part 2: Embeddings and semantic search

4.1 Loading documents

The project ships with five sample documents in `data/sample_docs` and indexes them automatically the first time the store is empty. Users can add more files of type PDF, TXT or Markdown from the interface; PDFs are read page by page with `pypdf`.

Document	Topic
<code>python_basics.md</code>	Python syntax, lists, tuples, dictionaries, functions
<code>machine_learning.md</code>	Supervised and unsupervised learning, overfitting, language models
<code>rag_explained.md</code>	Retrieval-Augmented Generation and why it reduces hallucination
<code>embeddings_guide.txt</code>	Embeddings, sentence transformers, cosine similarity, chunk overlap
<code>vector_databases.md</code>	ChromaDB, FAISS and the role of a vector store in RAG

4.2 Chunking

Whole documents are too long to embed as a single vector, because one vector would blur many topics together. Each document is therefore divided into chunks of about 500 characters. The splitter prefers to cut at the end of a sentence, then at a space, so that words are not broken. Consecutive chunks overlap by 50 characters; without overlap, a sentence that straddles a boundary would be cut in half and neither chunk would carry its full meaning.

4.3 Embeddings

Embeddings are lists of numbers that represent the meaning of a text. Texts with similar meaning are placed close together in the vector space, even if they share no words. Lexicon uses **all-MiniLM-L6-v2**, a sentence-transformer that produces 384-dimensional vectors, runs quickly on a CPU and gives strong results for semantic search. The model is executed through ONNX Runtime (the `fastembed` library) instead of PyTorch, which keeps the installation small enough for serverless hosting while producing the same kind of vectors.

4.4 Vector storage in ChromaDB

Each chunk is stored in ChromaDB with its text, its vector and metadata (source file name and chunk number). The collection is configured with cosine distance. Locally the database is persisted to disk; on Vercel, where the filesystem is read-only, an in-memory client is used and the sample documents are re-indexed on a cold start. Uploading a file with an existing name replaces its earlier chunks.

4.5 Retrieval

To answer a question, Lexicon embeds the question with the same model and asks ChromaDB for the three nearest chunks. Chroma returns cosine *distance*; the application also reports *similarity*, defined as 1 minus distance.

Value	Meaning
Similarity close to 1	The chunk and the question point in almost the same direction: a strong semantic match.
Similarity close to 0	The texts are unrelated.
Distance = 1 - similarity	Lower distance means a better match, and results are sorted by ascending distance.

The interface displays the three chunks as cards showing the source file, a similarity bar, the similarity score and the distance, so the user can see exactly which text the answer was based on.

5. Semantic search versus keyword search

Keyword search looks for the words of the query inside the text. A document is relevant if it contains the same words, and the ranking depends on how many words match. It is fast, predictable and excellent for exact names, error codes or phrases, but it has no understanding of language: it cannot connect "car" with "automobile" or "fix" with "repair".

Semantic search compares meaning rather than spelling. The query and every chunk are converted to embeddings, and chunks are ranked by how close their vectors are. A query such as "how do I fix my car?" can retrieve a passage about repairing an automobile although the two share no words, because the embedding model has learned from large amounts of text that these expressions are used in the same situations.

Aspect	Keyword search	Semantic search
Matching basis	Exact words in the query	Meaning, through embeddings
Synonyms and paraphrases	Missed unless the word appears	Handled naturally
Ranking signal	Number of shared words	Cosine similarity of vectors
Typical strength	Exact names, IDs, quoted phrases	Natural-language questions
Typical weakness	Fails when the wording differs	May return related but not exact text; needs an embedding model
Cost	Very cheap	Embedding computation and a vector index

Table 2. Comparison of the two approaches.

The Semantic vs keyword tab of the application runs both searches on the same query and shows the results side by side, with the matched words listed for keyword results and the similarity and distance for semantic results. In practice, many production systems combine both (hybrid search); for question answering over documents, semantic retrieval is the better default.

6. Implementation details

6.1 Project structure

```

lexicon-docqa/
|-- api/index.py          Vercel endpoint
|-- app/
|   |-- main.py           API routes
|   |-- config.py         settings
|   |-- schemas.py        request models
|   |-- prompts/templates.py zero-shot, few-shot, role-based
|   |-- services/
|       |-- chunking.py    text splitting
|       |-- embeddings.py  sentence-transformer
|       |-- vector_store.py ChromaDB, semantic + keyword search
|       |-- llm.py         Groq answers and judge scoring
|       |-- loaders.py     PDF / TXT / MD readers
|-- data/sample_docs/     5 documents
|-- public/               index.html, css/, js/
|-- tests/                pytest
|-- docs/                 report and presentation
`-- vercel.json, firebase.json, requirements.txt

```

6.2 API endpoints

Endpoint	Purpose
GET /api/health	Liveness check
GET /api/docs-list	Indexed documents and their chunk counts
POST /api/upload	Upload and index PDF, TXT or MD files
POST /api/ask	Retrieve top-3 chunks and answer with a chosen prompt mode
GET /api/search?q=	Semantic and keyword results for one query
POST /api/benchmark	Run all questions through all three prompts and score them

6.3 Frontend

The interface is a dependency-free single-page application in HTML, CSS and JavaScript. It has three tabs: **Ask** (question box, prompt selector, answer and retrieved chunks), **Compare prompts** (the five-question experiment with a score table and every answer) and **Semantic vs keyword** (side-by-side retrieval). It supports light and dark themes and adapts to small screens.

7. Deployment

The project is prepared for two hosts. **Vercel** runs both the FastAPI backend (as a Python function through `api/index.py`) and the static frontend from the `public` folder. **Firebase Hosting** can optionally serve the frontend alone, by setting the backend address in `public/js/config.js`. Firebase Hosting cannot run a Python server, so the backend always stays on Vercel.

- Push the repository to GitHub and import it in Vercel.
- Add the environment variable `GROQ_API_KEY` (optionally `GROQ_MODEL`) and deploy.
- For Firebase, set `window.API_BASE` to the Vercel URL, then run `firebase init hosting` and `firebase deploy`.
- Vercel raised the Python function bundle limit to 500 MB; using ONNX embeddings rather than PyTorch keeps the deployment well within it.

Live links: Frontend (Firebase Hosting): <https://lexicon-docqa.web.app> | Backend and API documentation (Vercel): <https://lexicon-docqa.vercel.app> | Source code: <https://github.com/scientistayat-engineer/lexicon-docqa>

8. Testing and limitations

Automated tests (pytest) cover the chunking function and the API's health and input validation. The retrieval and generation paths were designed to be exercised through the application itself, using the Compare prompts and Semantic vs keyword tabs.

Known limitations

- On Vercel the vector store is in memory: uploaded files last only while the instance is warm and the first request after idle time is slow because the model must load.
- Uploads are limited to about 4.5 MB per request on Vercel.
- Scores in the comparison come from an LLM judge, which is useful for relative comparison but is not a human evaluation and can vary slightly between runs.
- Scanned PDFs without a text layer are not supported because there is no OCR step.

Future work

- Persistent storage (a hosted vector database) so uploads survive restarts.
- Hybrid search that merges keyword and semantic scores, and a re-ranking step.
- Streaming answers and per-user document collections.

9. Conclusion

Lexicon demonstrates the full RAG workflow: documents are chunked, embedded and stored in ChromaDB; questions are matched to the most similar chunks by semantic search; and a language model answers from that evidence using a carefully designed prompt. The comparison of zero-shot, few-shot and role-based prompting shows that how a question is framed changes the quality of the answer even when the retrieved evidence is identical, and the semantic versus keyword comparison shows why embeddings are the right foundation for question answering over documents.

Appendix: running the project

```
python -m venv venv
venv\Scripts\activate      (Mac/Linux: source venv/bin/activate)
pip install -r requirements.txt
copy .env.example .env      (add GROQ_API_KEY)
uvicorn app.main:app --reload
```

Open <http://127.0.0.1:8000>

Run tests: `pip install -r requirements-dev.txt && pytest`